invested in JavaScript, it isn't a stretch to use it on the sever side as well. By using JavaScript on both the client and server it becomes trivial to share pieces of code that are common to both (such as form validation).

If you're familiar with JavaScript, Node.js is just a stone-throw away. The language is the same, so all you need to learn are some of the conventions and features of the Node.js environment.

Getting Up and Running with Node.js

Getting a Node.js running on your OS is quite a simple matter. All you need to do is visit the Node.js website, nodejs.org and download the latest version for your OS. You're likely to see two versions available, an LTS version for people who want long-term support. You're better off with the Stable version; it has the latest features.

Linux user might find their distro's package repositories have a rather old version. The Node.js website offers Linux users tar packages that you can just extract and run. Mac and Linux users can look into nvm or Node Version Manger that helps you keep your Node.js version up to date.

Once it's installed, you can type node at your command prompt to get a JavaScript REPL. So go ahead and try the obligatory "Hello World" program; type:

```
> console.log("Hello World");
```

You should see "Hello World" printed on the console, but you will also notice that Node.js prints out undefined after that. This is expected behaviour since Node.js will print the result of whatever code you type. If you typed 2 + 2 you'd see the result 4 printed at the console. The console.log function returns nothing and as such you see undefined.

If you end up doing something interesting in the REPL, you can save the code you've run using .save filename.js and later load it using .load filename.js

Modular JavaScript

Any serious application is likely to be spread across multiple files that each focus on a single aspect of the program. As such any serious programming language needs some form of module system that allows you to break down a large application into smaller pieces and put it together in some way.

Unfortunately JavaScript doesn't have a module system because of its origin as a browser-only language. Node.js has had to create its own system of modules that allows one JavaScript file to directly reference another, and to pull variables, and functions from it. There is a module system that is being standardised for JavaScript, and it isn't too hard to pick up once you are familiar with Node.js.

Let's say we want to create a simple program that inputs a user's name and outputs it in upper case; for this app we need some way to read user input. For our purpose we can use the inbuilt module 'readline' which provides this functionality, and we can get access it by setting a variable to require('readline'). The 'readline' functionality can now be accessed though this variable.

Here is the code in question:

```
const readline = require('readline');
const rl = readline.createInterface({
 input: process.stdin,
 output: process.stdout});
rl.question('name: ', (answer) => {
 console.log(answer.toUpperCase());
 rl.close();});
```

To run this code, put it in a file with the .js extension, and then run it as node mycode.js.

Like other scripting languages, Node.js comes with its own standard library. Given its focus on server-side applications, the functionality provided by Node.js is skewed towards the requirements of a web service.

Of course you are not limited to the libraries that Node.js ships with, there is an entire ecosystem of packages available for Node.js that is just a single command away.

Node Packages

Just like Ruby has gem, and Python has pip to install packages from a repository, Node.js has npm. Hundreds of thousands of packages are just a single command away. Just run npm install package_name in the directory of your project and you will be able to import its code and use it in your project.

ASIDE: While it may seem most obvious, npm does not, in fact, stand for Node Package Manager.

By default npm installs packages into a folder called node_modules in the directory in which you run it. This allows only code contained within this directory to use these modules. For some kinds of packages, such as command line tools it can be useful or even necessary to install them globally by adding a --global parameter. For instance Microsoft's command-line tool for managing their Azure cloud computing service can be installed with npm install --global azure-cli, after which it will be available to run as azure like any other command line tool.

Creating your own package is very simple with Node.js. Run npm init in the directory in which you want to initialise your project, and answer the questions it asks. This will create a package.json file in the folder. You can modify this file to your liking using any text editor.

Among other things this file lets you list all the other modules your code depends on, and the version of that package you need. You can create these entries manually or, add the --save parameter while installing to automatically install the package and add it as a dependency to the package.json file. This file can also list dependencies on other modules that might be needed only during development. For instance you might need JavaScript linting and testing tools like eslint and mocha while developing your application, but they aren't needed to run it. These can be installed using --save-dev instead of --save.

If a folder has a package.json file in it, you can install all the dependencies it lists by simply running npm install in that folder. These packages can also all be upgraded using npm upgrade. These upgrades always ensure not to upgrade to a version that might not by compatible any more.